

Vision-Based Target Following for a JetBot Robot Using OpenCV and a PSO-Like Controller

Abdalrahman Mohammed^{a,1} and Casper Aborres^{a,2}

^aM.Sc. Programme in Artificial Intelligence and Machine Learning, Blekinge Institute of Technology

This report was compiled on July 2, 2026

Abstract

This report presents a vision-based target-following system for multiple JetBot robots. The goal of the project was to make the robots detect and follow a red target using camera input, colour-based image processing and motor control. Instead of using a trained AI model, the system used HSV colour segmentation and pixel counting to identify the target position in the camera image. A client-server structure was implemented where each JetBot ran a local HTTP server for camera and motor access, while a client computer handled image processing, control logic and communication between robots. A PSO-like visual controller was used to allow robots to share target information through secondary colours, so that robots without direct line of sight to the red target could follow another robot that had already found it. The system was tested in three scenarios involving visible targets, blocked line of sight and a moving target. The results showed that the JetBots were able to detect, follow and cooperate towards the red target, although the system was limited by lighting sensitivity, lack of obstacle avoidance and no exact distance estimation.

Keywords: JetBot, computer vision, colour detection, HSV segmentation, pixel counting, robot control, swarm robotics, PSO-like controller, client-server communication

1. INTRODUCTION

With the increasing use of autonomous systems and automation in industry and research, robotic systems must be able to perceive and respond to their environment. This can be achieved using different types of sensors, including cameras, distance sensors and combinations of several sensing methods. At the same time, it is useful to investigate what can be achieved with a simple sensor setup and a simple control method. In this project, a vision-based control system was implemented on JetBot robots to study how camera input and motor control can be used to detect and follow a target object without using an artificial intelligence model.

JetBots are compact mobile robots based on the NVIDIA Jetson Nano platform. They include a camera, motor drivers and onboard computing power, making them suitable for experiments in computer vision, autonomous control and robotics education. Although the platform is capable of running AI-based models, this project used a simpler approach based on image processing and pixel counting.

The main goal of the project was to make multiple JetBots detect and track a coloured object, specifically a red box. This was achieved using a colour-based pixel detection method, where camera images were processed to identify pixels within a predefined red colour range. Instead of using a trained AI model, the system counted the number of matching pixels in different parts of the image to estimate the target position relative to the robot. Based on this information, the robots adjusted their angle to move towards and follow the target. In addition, a tracking function and a communication system were implemented to allow the robots to cooperate and share information.

This report describes the robot platform, the software structure, the target detection method, the control algorithm, the testing process, the observed behaviour and possible improvements.

2. GOALS

The goal of this project was to develop a JetBot robot system that could autonomously detect and follow a red target using camera-based colour detection and motor control. The system was also designed to support multiple JetBots, where robots could communicate and use each other as secondary targets if the red target was not directly visible.

The main goals of the project were:

- to stream camera images from the JetBot to a client computer,

- to detect a red target using OpenCV and HSV colour segmentation,
- to estimate the target position and direction error from the camera image,
- to convert visual information into motor commands,
- to make the robot search for the target when it was not visible,
- to make the robot stop when it became close to the target,
- to allow multiple robots to communicate when the red target was blocked,
- to evaluate the behaviour of the implemented robot system.

3. ROBOT PLATFORM AND TOOLS

3.1. JetBot Specifications

The robot platform used in this project was a JetBot robot. JetBot is a small differential-drive robot equipped with an IMX219-160 camera, two drive motors and an onboard computer. The robot can move forward, turn left, turn right and stop by controlling the speed of the left and right motors.

Although the JetBot platform is capable of running AI models, this project did not use an AI model. Instead, the target detection was based on simple colour filtering and pixel counting.

3.2. Sensors and Actuators

The main sensor used in the project was the JetBot camera. The camera provided image frames that were used for target detection. The image used in the program was 1000 by 1000 pixels. Before being sent to the client, the image was vertically cropped by 300 pixels. The larger image resolution was used to improve detection of objects at longer ranges, while the cropping was done to reduce unnecessary information from the environment and to reduce the amount of data sent over the network.

The actuators used in the project were the two drive motors of the

Abdalrahman Mohammed: abmm22@student.bth.se.
Casper Aborres: caab22@student.bth.se.

This report was written for a robotics course. It describes a JetBot system that detects and follows a red target using camera vision, colour-based pixel detection and a PSO-like visual control method. The system also uses communication between multiple robots to support cooperative target tracking.

JetBot. The motors were controlled using differential-drive commands. This means that the robot turns by giving different speeds to the left and right motors. For example, if the left motor speed is higher than the right motor speed, the robot turns to the right. If both motors are given the same speed, the robot moves forward. If the motors are stopped, the robot stops.

3.3. Software Environment

The project was implemented in Python. The main libraries and tools used were:

- `OpenCV` for image processing and colour detection,
- `NumPy` for numerical calculations,
- `requests` for HTTP communication between the client and the JetBot,
- `PIL` for converting received camera images,
- `http.server` for implementing the JetBot server,
- `threading` for running the server in a separate thread,
- JetBot's `Camera` and `Robot` classes for camera access and motor control.

4. METHOD AND IMPLEMENTATION

4.1. Overall System Design

The system was built to support multiple JetBots. To accomplish this, the code was divided into two main parts: a server side and a client side. The server side was implemented and run on each JetBot by hosting an HTTP server that listened for instructions, such as sending a camera image or stopping the robot. The client side handled the remaining parts of the system, such as object detection, movement logic, tracking logic and communication between robots.

The overall system structure is shown in Figure 1. Each JetBot ran the server code on the Jetson Nano, while the client computer ran the image processing and control logic. Communication between the client computer and the JetBots was done over Wi-Fi using HTTP requests.

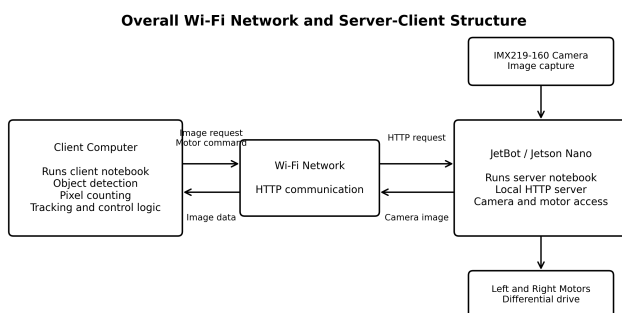


Figure 1. Block diagram of the Wi-Fi network and the server-client structure.

The system was designed as a real-time feedback loop. Camera information was continuously requested from the JetBot and processed by the client. Based on the detected target position, new motor commands were calculated and sent back to the robot. This made it possible for the robots to react to changes in their environment while the program was running.

The control process is shown in Figure 2. The flowchart shows how the camera image was processed using colour filtering and pixel counting, and how the PSO-like controller generated movement commands based on the detected target position.

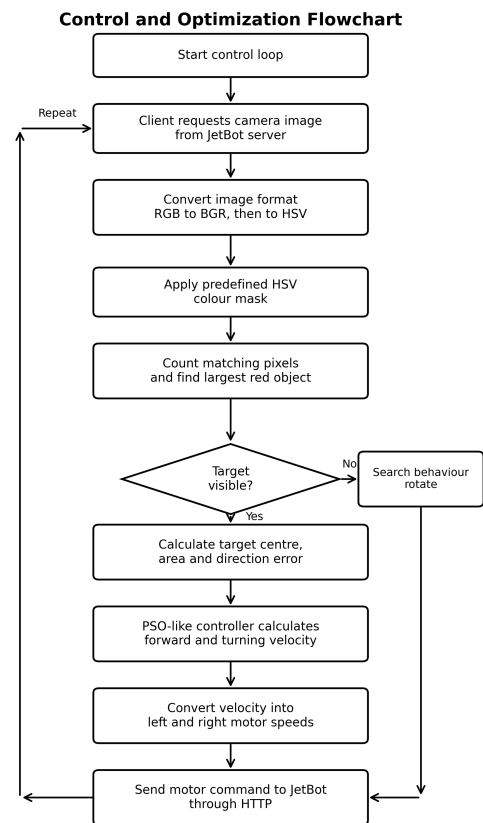


Figure 2. Flowchart of the control and optimization scheme.

The general control loop can be summarized as follows:

1. The client requests an image from the server running on the JetBot.
2. The image is converted into a format suitable for OpenCV processing.
3. A predefined HSV colour mask is applied to detect the red target.
4. Matching pixels are counted and the largest red object is selected.
5. The target centre, area and direction error are calculated.
6. The PSO-like controller calculates the forward and turning velocity.
7. The velocity values are converted into left and right motor speeds.
8. The motor command is sent from the client to the JetBot through HTTP.

4.2. Client-Server Communication

The JetBot server was responsible for giving access to the camera and the motors. The server used Python's `HTTPServer` and handled different HTTP paths. The `/camera` endpoint returned the current camera image as a JPEG image. The `/set_motors` endpoint received the left and right motor speeds and applied them to the robot.

The client used HTTP requests to communicate with the JetBot. For example, the client called the camera endpoint to read one image frame and called the motor endpoint to set the speed of the left and right motors. In Code 1, it can be seen how the client sends a request to a given JetBot IP address through the `/camera` endpoint. The received image is opened with `PIL` from bytes and converted to RGB format. It is then converted into a NumPy array and finally converted to BGR format using `cv2`, since OpenCV uses BGR as its default image format.

```
# Example of how request from camera looks
response = requests.get(f"http://{jetbot_ip}:8080/camera")
```

```

pil_image =
↳ PILImage.open(BytesIO(response.content)).convert("RGB")
frame_rgb = np.array(pil_image)
frame_bgr = cv2.cvtColor(frame_rgb, cv2.COLOR_RGB2BGR)

```

Code 1. Client-side request and image conversion from the JetBot server.

```

display(output_image)

# print(counts)
# print(detections)

time.sleep(0.05)

```

Code 2. Display average colour in a predefined area.

4.3. Colour-Based Target Detection

The target detection was based on colour segmentation in the HSV colour space. HSV was used because it separates hue, saturation and brightness, which makes colour thresholding easier than using the RGB colour space directly. It also helped reduce problems caused by changing light levels, uneven lighting in the environment and outside light from the sun.

To calculate the HSV ranges for the different colours, a helper function was implemented, as shown in Code 2. The function displayed a square on the camera image and printed the average colour value inside the square. This made it possible to hold the desired colour in front of the robot at different angles and distances, and then use the measured values to choose upper and lower HSV bounds.

```

def display_camera_stream(jetbot_ip):
    while True:
        response = requests.get(f"http://{jetbot_ip}:8080/camera")

        pil_image =
↳ PILImage.open(BytesIO(response.content)).convert("RGB")
        frame_rgb = np.array(pil_image)
        frame_bgr = cv2.cvtColor(frame_rgb, cv2.COLOR_RGB2BGR)

        height, width, _ = frame_bgr.shape

        # ----- SAMPLE TARGET COLOR -----

        box_w = int(width * 0.03)
        box_h = int(height * 0.12)

        box_w = max(box_w, 20)
        box_h = max(box_h, 20)

        x = width // 2 - box_w // 2
        y = height // 2 - box_h // 2
        w = box_w
        h = box_h

        crop = frame_bgr[y:y+h, x:x+w]

        crop_hsv = cv2.cvtColor(crop, cv2.COLOR_BGR2HSV)
        average_hsv = crop_hsv.reshape(-1, 3).mean(axis=0)

        cv2.rectangle(frame_bgr, (x, y), (x + w, y + h), (255,
↳ 255, 255), 2)

        cv2.putText(
            frame_bgr,
            f"HSV: {average_hsv.astype(int)}",
            (max(x - 60, 0), max(y - 10, 20)),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.5,
            (255, 255, 255),
            2
        )

        # -----

        detected_frame, counts, detections =
↳ detect_multiple_colors(frame_bgr)

        detected_rgb = cv2.cvtColor(detected_frame,
↳ cv2.COLOR_BGR2RGB)
        output_image = PILImage.fromarray(detected_rgb)

        clear_output(wait=True)

```

Five different masks were created. One red mask was used for the main target box, and one mask was created for each robot used in the project. The robot colours were blue, purple, yellow and orange. The red box was set as the main target colour, while the other colours were used as secondary target colours for identifying the robots.

The secondary target colours were used to support cooperation between the robots. A secondary colour was only targeted if the robot carrying that colour had detected the main red target. This allowed robots without a direct line of sight to the red box to move towards another robot that had already found the target. However, the red box remained the main target throughout the system. If the red box became visible again, the robot switched back to tracking the main target instead of the secondary target. The method used to select a secondary target is described later in the report.

4.4. Direction Estimation and Stopping Condition

The controller mainly used the horizontal position of the detected target to decide the movement direction of the robot. A separate distance estimation method was considered during the project, but it was not included in the final implementation. One reason for this was that the camera image was cropped before being sent to the client. This made distance estimation less reliable at shorter distances, since the full target could sometimes be partly outside the processed image area. In addition, adding distance estimation would have increased the complexity of the system. Therefore, the final controller used the detected target area as a simpler indication of how close the robot was to the target.

The horizontal direction was estimated from the centre position of the detected bounding box. The centre of the target was calculated as:

$$x_{target} = x + \frac{w}{2}$$

where x is the left position of the bounding box and w is the width of the bounding box. The centre of the camera image was calculated as:

$$x_{center} = \frac{W}{2}$$

where W is the width of the image frame. The horizontal error was then calculated as the difference between the target centre and the image centre:

$$e_x = x_{target} - x_{center}$$

A negative value means that the target is to the left of the image centre, while a positive value means that the target is to the right. If the error is close to zero, the target is considered to be centred.

To avoid unnecessary turning when the target was almost centred, a tolerance region was used around the centre of the camera image. In the final implementation, this tolerance was set to 25% of the image width. This meant that the robot did not need the detected target to be exactly in the centre of the image before moving forward. Instead, the robot moved forward as long as the target was within the centre tolerance region. This helped reduce small left and right corrections and made the movement more stable.

The tolerance was calculated as:

$$t = \frac{W \cdot p}{2}$$

where W is the image width and p is the tolerance percentage. In the final implementation, $p = 0.25$, meaning that the accepted centre region covered 25% of the image width.

The tolerance check was implemented using the function shown in Code 3. The function first calculates the centre position of the detected bounding box and the centre position of the camera image. It then calculates the allowed tolerance in pixels. If the distance between the target centre and the image centre is smaller than this tolerance, the target is considered centred.

```
def box_is_centered(box, frame_width, tolerance_percent=0.25):
    box_center_x = box["x"] + box["w"] // 2
    camera_center_x = frame_width // 2

    tolerance_pixels = frame_width * tolerance_percent / 2

    return abs(box_center_x - camera_center_x) <=
        tolerance_pixels
```

Code 3. Function used to check if the detected target is centred in the camera image.

The direction logic can be summarized as:

- if $|e_x| \leq t$, the target is centred and the robot moves forward,
- if $e_x < 0$, the target is to the left and the robot turns left,
- if $e_x > 0$, the target is to the right and the robot turns right.

Instead of estimating the exact distance to the target, the system used the detected pixel area of the target. When the detected area became larger than a predefined threshold, the robot assumed that it had reached the target and stopped. In the implementation, this threshold was defined by the parameter `stop_px`. In the final version, `stop_px` was set to 75000 pixels. This method was simpler than camera-based distance estimation and worked better with the cropped image used in the project.

4.5. PSO-Like Visual Controller

The movement controller was designed as a PSO-like visual controller. It was not a full mathematical implementation of Particle Swarm Optimization, but it was inspired by the idea that robots can use both their own target information and shared information from other robots. In this project, each robot tried to follow the main target, which was the red box. If the red box was not visible, the robot could instead follow a secondary target colour from another robot that had already detected the red target.

The controller used the result from the colour detection system to decide which target should be followed. The main target colour was always red. Each robot also had its own colour, for example blue, purple, yellow or orange. When a robot detected the main target, its own colour was added to a shared list of target colours. This meant that other robots could use that robot as a secondary target if they could not see the red box themselves.

The shared target colours were stored in the list `shared_target_colors`. Since several robot loops were running at the same time, a thread lock was used when reading or changing this list. This prevented several threads from modifying the shared list at the same time. The helper functions used for this are shown in Code 4.

```
#List shared between robots

shared_target_colors = []
target_lock = threading.Lock()

def add_target_color(color):
    with target_lock:
        if color not in shared_target_colors:
            shared_target_colors.append(color)

def get_shared_target_colors():
```

```
    with target_lock:
        return shared_target_colors.copy()

def remove_target_color(color):
    with target_lock:
        if color in shared_target_colors:
            shared_target_colors.remove(color)
```

Code 4. Functions for adding, reading and removing shared target colours.

Each robot was controlled by its own loop. In every iteration, the robot first read the current shared target colours. The robot then filtered out its own colour from the list, so that it would not try to follow itself. After this, the function `follow_color_step` was called. This function selected the most suitable visible target, calculated the movement direction and sent motor commands to the JetBot.

The main robot loop is shown in Code 5. If the robot detected the main red target, or detected one of the shared secondary targets, its own colour was added to the shared target list. This created a chain-like behaviour where robots that had found the target could guide other robots towards it. If a robot lost the target for too long, its colour was removed from the shared list so that other robots would no longer follow outdated information.

The main robot loop also used a short delay of 0.05 seconds between iterations. This gave an approximate control update period of 50 ms, not including the time needed for image transfer and processing. A shorter delay made the robot react faster, but also increased the number of HTTP requests and the processing load on the client computer. During the segmented spin search, the robot rotated in steps of 0.2 seconds with a pause of 0.1 seconds between checks. This made the search behaviour more stable, since the robot had time to receive and process a new camera image between movements.

```
def robot_loop(
    jetbot_ip,
    robot_color,
    main_target="red",
    speed=0.12,
    turn=0.025,
    search_spin_speed=0.1,
    min_area=300,
    stop_px=75000,
    tolerance_percent=0.25,
    stop_event=None
):
    target_lost_count = 0
    target_lost_limit = 200

    try:
        while stop_event is None or not stop_event.is_set():

            # Get newest shared target colors
            current_target_colors = get_shared_target_colors()

            # Do not follow own color
            current_target_colors = [
                color for color in current_target_colors
                if color != robot_color
            ]

            status, selected_color = follow_color_step(
                jetbot_ip,
                main_target=main_target,
                target_colors=current_target_colors,
                robot_color=robot_color,
                speed=speed,
                turn=turn,
                min_area=min_area,
                stop_px=stop_px,
                tolerance_percent=tolerance_percent
            )

            log_message(jetbot_ip, "status:", status,
                "selected:", selected_color)

            # ----- CHAIN LOGIC -----
```

```

if selected_color is None:
    target_lost_count += 1

elif selected_color == main_target or selected_color
↳ in current_target_colors:
    target_lost_count = 0
    add_target_color(robot_color)

else:
    target_lost_count += 1

if target_lost_count >= target_lost_limit:
    remove_target_color(robot_color)

# -----

if status == "reached":
    stop_robot(jetbot_ip)
    log_message(jetbot_ip, "reached target")
    break

if status == "searching":
    log_message(jetbot_ip, "starting segmented spin
↳ search")

    box, selected_color = spin_until_color_found(
        jetbot_ip,
        main_target=main_target,
        robot_color=robot_color,
        spin_speed=search_spin_speed,
        min_area=min_area,
        tolerance_percent=tolerance_percent,
        step_time=0.2,
        pause_time=0.1,
        stop_event=stop_event
    )

    log_message(jetbot_ip, "spin result:",
↳ selected_color)

    # Get newest target list AFTER spinning
    latest_target_colors = get_shared_target_colors()

    latest_target_colors = [
        color for color in latest_target_colors
        if color != robot_color
    ]

    if selected_color is None:
        target_lost_count += 1

    elif selected_color == main_target or
↳ selected_color in latest_target_colors:
        target_lost_count = 0
        add_target_color(robot_color)

    else:
        target_lost_count += 1

    if target_lost_count >= target_lost_limit:
        remove_target_color(robot_color)

    continue

time.sleep(0.05)

finally:
    remove_target_color(robot_color)
    stop_robot(jetbot_ip)
    log_message(jetbot_ip, "stopped")

```

Code 5. Main robot loop for the PSO-like visual controller.

If no valid target was visible, the robot entered a search behaviour. In this case, the robot stopped following and started a segmented spin search. During this search, the robot rotated in small steps while checking if either the main target or a valid secondary target became visible. If a target was found, the robot restarted the loop and continued following based on the newest camera image and the newest

shared target list.

5. RESULTS

5.1. Evaluating the System

To evaluate the functionality of the JetBot system, three different test scenarios were created. The purpose of the tests was to see if the main goals of the project were achieved. These goals were to identify and track the red target, to react to changes in the target position and to use communication between robots to help robots without direct line of sight reach the target.

In the first scenario, a red box was placed in the environment and the robots were placed around it in different starting directions. Some robots were facing the box, while others were turned away from it. The result of this test was that the robots that did not initially see the red target started searching by rotating. When the red target was detected, the robots started moving towards it. All robots were able to locate and move towards the red box.

In the second scenario, some robots had their line of sight to the red box blocked. These robots were therefore required to use other JetBots, which had already detected the red box, as secondary targets. This tested the communication and cooperation part of the system. The result was that the robots without direct line of sight were able to follow the secondary target colours and move towards the red objective indirectly.

In the third scenario, the JetBots were first placed with line of sight to the red box. While the system was running, the red box was moved to test the responsiveness and flexibility of the controller. The robots successfully changed their direction and adjusted their movement when the red target position changed.

Overall, the tests showed that the system was able to detect the red target, guide the robots towards it and use secondary target colours when direct line of sight was not available. The system also reacted to changes in the target position while running.

5.2. Test Result Table

The results from the physical tests are summarized in Table 1.

Table 1. Test results for the JetBot target-following system.

Scenario	Trials	Successful	Success Rate
Robots placed around visible red target	1	Yes	100%
Some robots without direct line of sight	1	Yes	100%
Red target moved during operation	1	Yes	100%

5.3. Figures and Screenshots

Figures 3 and 4 show examples of the physical test setups used to evaluate the JetBot system. These screenshots were used to document how the robots and the red target were placed during testing. Scenario 2 tested the system when the red target was partly blocked, while Scenario 3 tested how the robots reacted when the red target was moved during operation.



Figure 3. Scenario 2 setup, where the red target was placed behind the box to block direct line of sight for some robots.



Figure 4. Scenario 3 setup, where the red target was visible at the start and later moved during operation.

6. DISCUSSION

6.1. Performance Analysis

The results shown in Table 1 indicate that the JetBot system met the main performance requirements of the project. The system was designed to detect and follow a red target, react to changes in the target position and use communication between robots when direct line of sight was not available. Based on the performed tests, the system was able to achieve these goals.

The robots were able to detect the red target using colour-based pixel detection and move towards it using the motor controller. When the target was not visible, the search behaviour allowed the robots to rotate until a valid target was detected. The system also showed that the secondary target colours could be used to guide robots that did not initially see the red box.

However, the tests were limited in number and were performed in a controlled environment. Therefore, the results should be seen as an indication that the system works, rather than a complete performance evaluation.

6.2. Strengths of the Implementation

One strength of the implementation was the clear separation between server and client responsibilities. The JetBot server handled camera access and motor control, while the client handled image processing, communication logic and decision-making. This made the program easier to test and modify, since changes to the detection and control logic could be made on the client side without changing the server code on each robot.

Another strength was that the detection method was simple and computationally efficient. Instead of using a trained AI model, the system used HSV colour thresholding and pixel counting. This made the method easier to understand, tune and debug. It also reduced the computational requirements, which is useful for small robotics systems and real-time control.

The use of secondary target colours was also a strength of the system. It allowed the robots to cooperate in a simple way without requiring a complex mapping system or global positioning. A robot that had detected the red target could become a guide for other robots, which made it possible for robots without direct line of sight to still move towards the target area.

The use of a tolerance region around the image centre also improved the movement behaviour. Since the robot did not need the target to be exactly centred before moving forward, the controller avoided unnecessary small corrections. This made the robot movement more stable and less sensitive to small changes in the detected target position.

6.3. Limitations

While the system had several strengths, it also had some limitations. One limitation that was not fully reflected in the system tests was obstacle handling. The robots did not have a method for detecting obstacles, avoiding them or freeing themselves if they became stuck. During testing, it could be seen that a JetBot sometimes got stuck with one wheel while still trying to continue forward, especially when following another moving JetBot.

A simple solution that was considered was to detect if the motors were working harder than normal, for example by measuring current or energy spikes. However, the JetBot setup used in this project did not include sensors for measuring motor load or wheel movement. Another possible solution would be to use a more advanced vision-based method or an AI model to detect obstacles, but this would increase both the complexity and the computational requirements of the system.

Another limitation was that changes in lighting could affect the colour detection. Uneven lighting, shadows or sunlight could change how the colours appeared in the camera image. This could lead to both false positives, where the system detected a colour that was not the target, and false negatives, where the real target was not detected. To reduce this problem, the colour masks were manually calibrated using the camera before running the system. The image was also cropped to limit the area where false positives could occur.

A final limitation was that no exact distance estimation was used in the final implementation. Instead, the system used the detected pixel area as a simple stopping condition. This limited the efficiency of the JetBot movement, since the robot could not estimate how far away the target was. However, distance estimation was not considered a core part of the project goal, so this was treated as a smaller issue compared to target detection, tracking and communication between robots.

7. CONCLUSION

The goal of this project was to develop a vision-based control system for multiple JetBots. The robots were designed to detect and follow a red target using only camera input, colour filtering and pixel counting. No trained AI model was used for the target detection. Instead, the system used simple image processing in the HSV colour space to identify the target and estimate its position in the camera image.

The final system was able to detect the red target, move towards it and react when the target position changed. The robots also used a simple communication method where robots that had detected the red target could share their own colour as a secondary target. This allowed other robots without direct line of sight to follow a guide robot and move towards the target indirectly.

The project showed that a relatively simple vision-based method can be used to create cooperative behaviour between multiple robots. The client-server structure made the system easier to test and modify, since the JetBot handled camera and motor access while the client handled detection, control and communication logic.

However, the system also had limitations. The robots did not have obstacle avoidance or a reliable method for detecting when they were stuck. The colour detection was also sensitive to lighting conditions, and no exact distance estimation was used in the final implementation. Future improvements could include obstacle detection, automatic colour calibration, better distance estimation and more extensive testing in different environments.

Overall, the project achieved its main goals. The JetBots were able to track a red target, cooperate using secondary target colours and respond to changes in the environment using a simple real-time feedback loop.

8. AI USAGE

AI tools were used during the writing process to help refine the text of the report. The AI was used to improve grammar, sentence structure and readability. It was also used to help organize some sections and make the explanations clearer.

■ APPENDIX

Appendix A: Main Code Files

The implementation consisted of several code files and notebooks:

- `server.ipynb`: runs on the JetBot and provides camera and motor endpoints.
- `client.ipynb`: communicates with the JetBot and contains earlier target-following functions.
- `detect.py`: contains colour detection functions for multiple colours.